T1 OpenMP: feedback geral para a turma

Este documento reúne o que apareceu com mais frequência nas entregas e apresentações do T1, para orientar os próximos trabalhos. Ele é publicado junto do enunciado do T2 e, onde a lição muda ao passar do OpenMP para o MPI no cluster, o ponto diz como. Nenhum grupo é identificado; cada um recebeu também um comentário individual no Moodle.

Como este foi o primeiro trabalho da disciplina, a correção foi mais tolerante: vários problemas que comprometem a validade dos resultados foram apontados, mas pesaram pouco na nota. **Nos próximos trabalhos os critérios do enunciado serão aplicados integralmente.** Os pontos abaixo são justamente os que vão fazer diferença.

O que funcionou bem

Vale começar pelo que as melhores entregas fizeram, porque é o padrão esperado daqui em diante:

- **Equivalência demonstrada, não afirmada.** O programa imprime, em toda execução e para todos os números de threads, um valor que resume o resultado (um checksum, a contagem de soluções, o número de iterações), e esse valor é sempre o mesmo da versão sequencial.
- **Previsão feita antes e confrontada com a medição.** Um exemplo: com `schedule(static)`, cada thread recebe um bloco fixo de iterações, e o programa só termina quando a thread com o bloco mais pesado termina. Medindo a carga de cada bloco, um grupo previu a eficiência como carga média dividida pela carga máxima (0,47) e depois confirmou esse valor na execução. Outros previram o tempo da escalabilidade fraca antes de medi-la, ou estimaram o ganho esperado em processadores que misturam núcleos rápidos (P) e econômicos (E).
- **Hipóteses testadas com ferramentas.** Prender cada thread a um núcleo específico com `taskset` para isolar um efeito, medir a frequência do processador e as instruções por ciclo com `perf stat`, contar operações para provar que o trabalho da escalabilidade fraca cresceu na proporção certa.
- **Dados brutos que conferem com o relatório.** Nos melhores trabalhos, recalcular as medianas a partir dos CSVs dá exatamente os números das tabelas.

Problemas mais frequentes e como evitá-los

1. Mostrar que as versões sequencial e paralela são equivalentes

Não basta dizer que o resultado "está certo". Registre uma grandeza verificável (checksum, contagem, soma, norma do erro) em **toda** execução e mostre que ela não muda. Quando o resultado paralelo não é idêntico bit a bit, por exemplo porque somas em ponto flutuante mudam de ordem, isso precisa ser explicado, e o critério de equivalência (uma tolerância) precisa ser definido e verificado. Se o resultado varia entre execuções e o grupo não sabe dizer por quê, a equivalência não foi demonstrada.

2. Entregar tudo o que é preciso para reproduzir

Um trabalho sem código não pode ser conferido. A entrega precisa conter: fonte, `Makefile` ou linha de compilação, **todos** os arquivos de entrada usados, tempos brutos e um `README` com os comandos exatos, incluindo opções que mudam o comportamento do programa (no T1, por exemplo, a variável que escolhia o escalonamento). Houve entregas sem código, com arquivo de entrada faltando e com a opção necessária para reproduzir a tabela não documentada. Se usar repositório, coloque o link no cabeçalho. Se partir de código de terceiros, cite a origem.

3. Seguir o protocolo de medição

O protocolo existe para que os números signifiquem alguma coisa:

- **10 repetições** por configuração, com **mediana, mínimo e máximo em todas as tabelas**, não só na principal.

- **Máquina ociosa**: sem navegador aberto e fora de máquinas virtuais ou WSL2. No cluster, o equivalente é rodar com `--exclusive`: sem isso outro grupo pode dividir o nó com vocês no meio da medição.
- **T_1, o tempo sequencial de referência, entre 30 e 60 s**, vindo da melhor versão sequencial, compilada com as mesmas flags e executada na mesma máquina da versão paralela. Se a versão paralela com 1 thread ficou mais rápida que a sequencial, há algo errado com a sequencial.
- **Mesma entrada em todos os pontos da escalabilidade forte**, e mesma série de threads (ou de processos) na forte e na fraca.

4. Manter os números consistentes entre si

Várias apresentações travaram em perguntas do tipo "esta tabela diz X, aquela diz Y para a mesma configuração; qual vale?". Antes de entregar, confira:

- o mesmo experimento aparece com o mesmo tempo em todas as tabelas e seções;
- se vocês testaram variações (outro escalonamento, outra granularidade), os gráficos principais usam a melhor delas, e não uma que as próprias tabelas mostram ser pior;
- a fração serial usada no Amdahl é a mesma ao longo do texto e é compatível com os dados;
- números de rodadas de medição diferentes (outro dia, outra versão do código) estão identificados, e a diferença entre eles está explicada.

5. Escalabilidade fraca

A escalabilidade fraca foi o ponto mais frágil da turma.

- O ponto de partida é **a mesma entrada de referência da forte**, com os mesmos parâmetros (iterações, passos, precisão). Só o tamanho cresce.
- A eficiência é T_1 da entrada inicial dividido pelo tempo com p threads e entrada p vezes maior. **Eficiência acima de 1 é sinal de erro**, quase sempre no T_1 usado como referência (medido com outra entrada ou com outra versão do programa).
- Crescer o parâmetro de entrada não garante que o **trabalho** cresça na mesma proporção. Verifique com uma medida de trabalho (iterações, nós visitados, operações) e mostre isso no relatório.

6. Perfil e limite de Amdahl

O perfil precisa vir de uma ferramenta (`gprof` ou `perf`), com a saída na entrega. Um percentual afirmado sem fonte não conta como perfil. O limite de Amdahl não é um número para constar no relatório e ser esquecido: ele serve para comparar com o speed-up medido. Quando a diferença é grande, **explicar essa diferença é a parte central da análise**. E se a fração serial cresce com o número de threads (porque o custo de criar e sincronizar threads entra nela), a lei de Amdahl com uma fração fixa não descreve o programa, e isso precisa ser dito.

No mestre/escravo do T2, o papel do Amdahl passa para o custo de comunicação. O mestre atende um escravo de cada vez: medindo o tempo de processar uma tarefa e o de uma ida e volta de mensagem, dá para prever a partir de quantos escravos o mestre não dá mais conta e passa a ser o gargalo. A lógica é a mesma do T1: previsão primeiro, depois a comparação com o medido.

7. Explicar causas com medição, não com suposição

Frases como "pode estar ligado a", "uma explicação possível é" e "provavelmente é a máquina virtual" indicam que a análise parou antes de terminar. Quando um resultado é inesperado (o tempo piora ao passar de 4 para 6 threads, o ganho fica muito abaixo do previsto, a execução mais lenta de uma configuração leva 60% a mais que a mais rápida), formule uma hipótese e **teste-a**: prenda threads a núcleos, meça a frequência, repita a execução, varie um parâmetro de cada vez. Quando o dado contradiz a hipótese, é a hipótese que deve mudar.

Uma pergunta útil: a causa está na aplicação (unidades de trabalho com custo irregular, acesso à memória, sincronização) ou no ambiente (máquina virtual, outros programas rodando)? Se está no ambiente, o certo é corrigir o ambiente e medir de novo, não explicar o número.

8. Dominar o próprio trabalho na apresentação

Na apresentação, cada integrante precisa saber explicar **todas** as diretivas e cláusulas do código (`nowait`, `schedule`, `reduction`, escopo de variáveis), cada gráfico e cada número das tabelas. No MPI, a pergunta passa a ser sobre cada chamada: de quem um `MPI_Recv` espera, com que tag, e o que acontece se a mensagem não vier. Houve casos de relatório tecnicamente bom com apresentação que não conseguiu explicar o gráfico da escalabilidade fraca ou uma cláusula usada no código; nos próximos trabalhos isso pesa na nota individual. Isso vale para qualquer código entregue, escrito à mão ou com apoio de IA.

Reconhecer os problemas do próprio trabalho na apresentação é melhor do que não percebê-los, mas o ideal é encontrá-los e corrigi-los antes da entrega.

9. Formato do relatório

Máximo de **duas páginas**, incluindo referências. Várias entregas passaram do limite, e isso vale como "fora do formato solicitado" a partir do T2. Cabeçalho com integrantes e link do repositório. Toda figura e tabela referenciada no texto. Uma tabela rotulada "mediana" traz medianas, não médias.

Antes de entregar o próximo trabalho

Uma lista curta para conferir, que teria evitado a maior parte dos problemas do T1:

- ☐ As versões sequencial e paralela produzem o mesmo resultado, e isso está **registrado** em cada execução.
- ☐ Alguém que não é do grupo consegue compilar e reproduzir as tabelas só com o que foi entregue e o `README`.
- ☐ Toda tabela tem mediana, mínimo e máximo de 10 repetições.
- ☐ A mesma configuração tem o mesmo tempo em todo o relatório.
- ☐ Nenhuma eficiência passa de 1 sem explicação.
- ☐ O trabalho da escalabilidade fraca cresce na proporção do número de threads ou processos, e isso foi medido.
- ☐ Toda previsão (Amdahl, razão entre computação e comunicação) foi comparada com o medido, e a diferença foi explicada.
- ☐ Cada conclusão da análise está apoiada em uma medição, não em "pode ser".
- ☐ Cada integrante sabe explicar cada diretiva ou chamada MPI e cada gráfico.
- ☐ O relatório tem no máximo duas páginas e o cabeçalho tem o link do repositório.